

Good examples should teach usable next steps.

To tackle an unfamiliar goal, a model must find missing information, choose smaller steps and use tools correctly.

Our test is TextCraft, a crafting game.

The model looks up recipes, makes parts and assembles the requested items. The game checks whether it made the right quantities, not whether its answer sounds convincing.

Our longer-term aim is a recursive language model: it can ask another model to solve a useful part of the task.

First, can better worked examples teach decisions that make sense from what the model has seen?

The teaching and reward-training results here use one small model family, Qwen3-4B. They do not establish that recursion helps.

Introduce a part before asking about it.

Supervised fine-tuning (SFT) teaches the model to copy the next action.

Original history

Goal: Make a pickaxe.
Part Z has not appeared yet.



Next action to learn

Ask for the part Z recipe.

Repaired history

Goal: Make a pickaxe.
Pickaxe lookup: it needs part Z.



The same action to learn

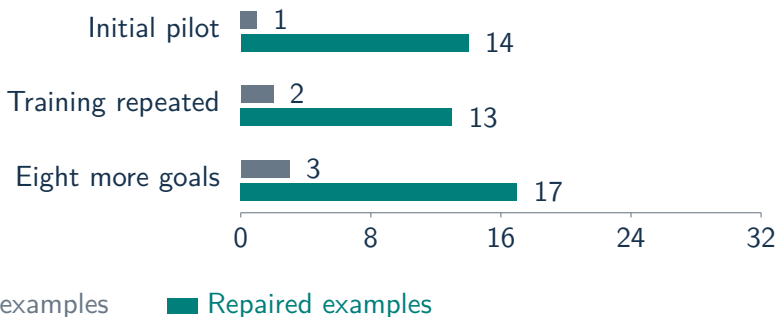
Ask for the part Z recipe.

The repair changes lookup order and input histories, not the taught answers or number of training steps.

Illustration only. The repair uses a known correct action list; it is not a discovery planner.

The repair helped in three small comparisons.

Successes out of 32; the taught answers and number of training steps stay fixed.



Each row repeats eight goals in two recipe worlds, twice per world. The first two rows reuse the same goals. These are exploratory, previously examined panels. Input lengths differ; extra training partly rescues the original examples. This tests limited training budgets.

Reward training helped, but cost more.

Reinforcement learning (RL) rewards task completion. We trained once from a batch of attempts, separately for each tool setting.

Tool setting	Successes before → after	Generated text
Model supplies ingredients	9 → 13 out of 16	+6%
Code fills known ingredients	14 → 16 out of 16	+22%

These are eight familiar training goals, with two new randomized attempts each. There is only one reward-training run per setting.

Tests on different goals and against more example-based training were still pending at this cutoff.

Code uses only recipes already looked up; it does not choose the item or quantity. Mistakes remain, and calls also increased. This is not yet efficient planning or demonstrated transfer.

Next, test transfer and the value of one helper.

Does the teaching repair work in another model family?

We will train a second kind of model, Phi-4-mini, on the repaired examples. That test is still pending.

Does a helper improve the whole task?

We will compare one helper with working alone, under the same total allowance for calls and generated text.

A helper completed a subtask, but the short test stopped before we could assess the whole task.

For discussion: Which result would most change your confidence in this approach?

No whole-task helper benefit or general recursive reasoning has been demonstrated.